



DAY 4

WEEK 2 — WEB EXPLOITATION

Authentication Flaws

What are Authentication Flaws?

Authentication is the process of verifying who you are. Authentication flaws are logic mistakes developers make that let attackers bypass login systems, reset passwords without authorization, or access accounts they don't own.

These vulnerabilities are often not about injecting code — they're about exploiting broken business logic. That makes them tricky and extremely common in the real world.

Common Authentication Vulnerabilities

1. Default / Weak Credentials

Many systems ship with default credentials that admins forget to change:

```
admin:admin
admin:password
admin:admin123
root:root
guest:guest
```

2. Username Enumeration

A poorly designed app might respond differently for valid vs invalid usernames:

Invalid user	Returns: 'User not found'
--------------	---------------------------

Valid user, wrong pass	Returns: 'Incorrect password'
---------------------------	-------------------------------

This allows an attacker to enumerate (discover) valid usernames before attacking passwords!

3. Broken Password Reset

- Predictable reset tokens: ?token=12345 — easy to guess/brute-force
- Token doesn't expire — attacker can use an old link
- Token sent in URL — logged by servers and browser history
- Security questions with guessable answers

4. No Rate Limiting / Brute Force Protection

```
# Hydra - HTTP brute force tool
hydra -l admin -P /usr/share/wordlists/rockyou.txt \
```

```
http-post-form '//login:username=^USER^&password=^PASS^:Invalid'
```

```
# Medusa
```

```
medusa -h target.com -u admin -P passwords.txt -M http -m DIR:/login
```

5. Insecure 'Remember Me' / JWT Issues

JWT (JSON Web Tokens) are commonly used for sessions. Common flaws:

- Algorithm set to 'none' — signature is skipped entirely
- Weak secret key — token can be forged offline
- Token stored in localStorage — accessible to XSS

```
// A JWT looks like three base64 parts:
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoiyWRtaW4ifQ.SflKxwRJSMeKKF2QT4fwpM
└─ Header ─┘ └─── Payload ───┘ └─ Signature ─┘

// Decode at: https://jwt.io
// Payload decodes to: {"user": "admin"}
// If signature isn't verified → change user to 'admin' and send it!
```

Tools

```
# Gobuster — directory brute force (find hidden login pages)
gobuster dir -u http://target.com -w
/usr/share/wordlists/dirbuster/medium.txt

# Burp Suite — intercept & modify login requests
# Enable proxy → Intercept → send to Intruder for automated attacks

# jwt_tool — test JWT vulnerabilities
pip3 install jwt_tool
python3 jwt_tool.py <token> -X a # Test 'alg: none' attack
```

Reference Links

- [PortSwigger Authentication Vulnerabilities Labs](#)
- [OWASP Broken Authentication Guide](#)
- [JWT Attack Cheat Sheet](#)
- [Hydra Brute Force Tool Guide](#)
- [PentesterLab — Auth Bypass Exercises](#)